

```
!pip install transformers datasets torch scikit-learn matplotlib gradio
```

```
Requirement already satisfied: transformers in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: datasets in /usr/local/lib/python3.12/dist-packages (4.0.0)
Requirement already satisfied: torch in /usr/local/lib/python3.12/dist-packages (2.10.0+cu128)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (5.50.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers) (3.24.2)
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (26.0)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers) (2025.11.3)
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.24.0)
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers) (0.7.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers) (4.67.3)
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1.0)
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.3.8)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.32.4)
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets) (0.70.16)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fsspec[http]<=) (2025.3.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4) (1.1.0)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0)
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (24.1.0)
```

```
import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import random

from transformers import AutoModel, AutoTokenizer
from datasets import load_dataset
from torch.utils.data import Dataset, DataLoader

from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
```

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from datasets import load_dataset

MODEL_NAME = "roberta-base"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)

# Step 2: Load dataset
```

```
dataset = load_dataset("squad", split="train[:2000]")
```

```
# Step 3: Load pretrained model ← THIS IS THE MISSING STEP
model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=2
)
```

/usr/local/lib/python3.12/dist-packages/huggingface\_hub/utils/\_auth.py:94: UserWarning:  
The secret `HF\_TOKEN` does not exist in your Colab secrets.  
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set  
You will be able to reuse this secret in all of your notebooks.  
Please note that authentication is recommended but still optional to access public models or datasets.

```
warnings.warn(
config.json: 100%                               481/481 [00:00<00:00, 46.9kB/s]
tokenizer_config.json: 100%                     25.0/25.0 [00:00<00:00, 1.78kB/s]
vocab.json: 100%                               899k/899k [00:00<00:00, 3.63MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN t
merges.txt: 100%                               456k/456k [00:00<00:00, 1.58MB/s]
tokenizer.json: 100%                           1.36M/1.36M [00:00<00:00, 3.76MB/s]
README.md: 7.62k/? [00:00<00:00, 291kB/s]
plain_text/train-00000-of-00001.parquet: 100% 14.5M/14.5M [00:00<00:00, 19.1MB/s]
plain_text/validation-00000-of-00001.par(...): 100% 1.82M/1.82M [00:00<00:00, 7.00MB/s]
Generating train split: 100%                   87599/87599 [00:00<00:00, 204683.85 examples/s]
Generating validation split: 100%              10570/10570 [00:00<00:00, 103850.53 examples/s]
model.safetensors: 100%                      499M/499M [00:03<00:00, 216MB/s]
Loading weights: 100%                        197/197 [00:00<00:00, 376.23it/s, Materializing param=roberta.encoder.layer.11.output.dense.weight]
```

RobertaForSequenceClassification LOAD REPORT from: roberta-base

| Key                             | Status     |
|---------------------------------|------------|
| lm_head.dense.bias              | UNEXPECTED |
| lm_head.layer_norm.weight       | UNEXPECTED |
| lm_head.layer_norm.bias         | UNEXPECTED |
| roberta.embeddings.position_ids | UNEXPECTED |
| lm_head.dense.weight            | UNEXPECTED |
| lm_head.bias                    | UNEXPECTED |
| classifier.out_proj.weight      | MISSING    |
| classifier.out_proj.bias        | MISSING    |
| classifier.dense.bias           | MISSING    |
| classifier.dense.weight         | MISSING    |

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task

```
def get_explanation(question, context, answer, top_k=5):
    model.eval()
    with torch.no_grad():
        enc = tokenizer(
            question,
            context + " " + answer,
            truncation=True,
            padding="max_length",
            max_length=256,
            return_tensors="pt"
        )

        input_ids = enc["input_ids"].to(device)
        attention_mask = enc["attention_mask"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        # Get attentions from last layer
        attentions = outputs.attentions[-1] # last layer
        attn_scores = attentions.mean(dim=1).squeeze() # average heads

        # Convert ids to tokens
        tokens = tokenizer.convert_ids_to_tokens(input_ids[0])
```

```

# Compute importance scores
scores = attn_scores.mean(dim=0).cpu().numpy()

# Get top important tokens
token_scores = list(zip(tokens, scores))
token_scores = sorted(token_scores, key=lambda x: x[1], reverse=True)

important_tokens = [
    tok for tok, _ in token_scores
    if tok not in ["[CLS]", "[SEP]", "[PAD]"]
][:top_k]

return important_tokens

```

```

data = []

for item in dataset:
    question = item["question"]
    context = item["context"]
    correct_answer = item["answers"]["text"][0]

    # correct sample
    data.append((question, context, correct_answer, 1))

    # incorrect sample (simple wrong answer)
    wrong_answer = correct_answer[:-1]
    data.append((question, context, wrong_answer, 0))

```

```

class QAVerificationDataset(Dataset):
    def __init__(self, data, tokenizer, max_len=256):
        self.data = data
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        question, context, answer, label = self.data[idx]
        text = question + " " + context + " " + answer

        encoding = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_len,
            return_tensors="pt"
        )

        return {
            "input_ids": encoding["input_ids"].squeeze(),
            "attention_mask": encoding["attention_mask"].squeeze(),
            "label": torch.tensor(label, dtype=torch.float)
        }

```

```

dataset = QAVerificationDataset(data, tokenizer)
loader = DataLoader(dataset, batch_size=8, shuffle=True)

```

```

class ConfidenceModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = AutoModel.from_pretrained(MODEL_NAME)

        self.classifier = nn.Sequential(
            nn.Linear(768, 256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, input_ids, attention_mask):
        outputs = self.encoder(
            input_ids=input_ids,

```

```

        attention_mask=attention_mask
    )
    cls_output = outputs.last_hidden_state[:, 0]
    logits = self.classifier(cls_output)
    return logits.squeeze()

```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```

model = ConfidenceModel().to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=2e-5)
loss_fn = nn.BCEWithLogitsLoss()

```

Loading weights: 100% 197/197 [00:01<00:00, 173.44it/s, Materializing param=encoder.layer.11.output.dense.weight]

RobertaModel LOAD REPORT from: roberta-base

| Key                             | Status     |
|---------------------------------|------------|
| lm_head.dense.bias              | UNEXPECTED |
| lm_head.layer_norm.weight       | UNEXPECTED |
| lm_head.layer_norm.bias         | UNEXPECTED |
| roberta.embeddings.position_ids | UNEXPECTED |
| lm_head.dense.weight            | UNEXPECTED |
| lm_head.bias                    | UNEXPECTED |
| pooler.dense.weight             | MISSING    |
| pooler.dense.bias               | MISSING    |

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task

EPOCHS = 3

```

for epoch in range(EPOCHS):
    model.train()
    total_loss = 0

    for batch in loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)

        optimizer.zero_grad()
        logits = model(input_ids, attention_mask)
        loss = loss_fn(logits, labels)

        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1} Loss: {total_loss:.4f}")

```

Epoch 1 Loss: 161.0636

Epoch 2 Loss: 98.7136

Epoch 3 Loss: 93.0378

```

torch.save(model.state_dict(), "confidence_model.pt")
print("Model saved successfully.")

```

Model saved successfully.

```
torch.save(model, "full_confidence_model.pt")
```

```

from google.colab import drive
drive.mount('/content/drive')

```

Mounted at /content/drive

```
import os

output_dir = "/content/drive/MyDrive"
# Create the directory if it doesn't exist
os.makedirs(output_dir, exist_ok=True)

torch.save(model.state_dict(), os.path.join(output_dir, "confidence_model.pt"))
```

```
model.eval()
all_logits = []
all_labels = []

with torch.no_grad():
    for batch in loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["label"].to(device)

        logits = model(input_ids, attention_mask)

        all_logits.extend(logits.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())
```

```
probs = torch.sigmoid(torch.tensor(all_logits)).numpy()
preds = (probs > 0.5).astype(int)
```

```
acc = accuracy_score(all_labels, preds)
f1 = f1_score(all_labels, preds)

print("Accuracy:", acc)
print("F1-score:", f1)
print(classification_report(all_labels, preds))
```

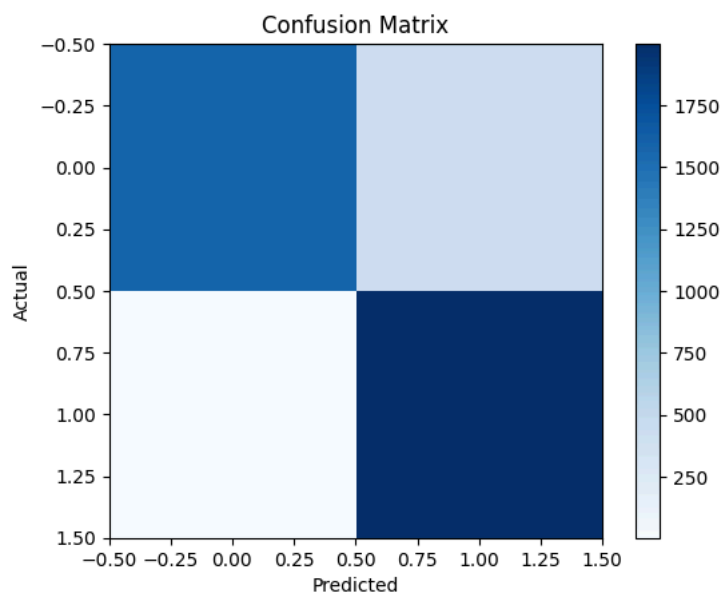
```
Accuracy: 0.89425
F1-score: 0.9042336427439438
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0.0          | 1.00      | 0.79   | 0.88     | 2000    |
| 1.0          | 0.83      | 1.00   | 0.90     | 2000    |
| accuracy     |           |        | 0.89     | 4000    |
| macro avg    | 0.91      | 0.89   | 0.89     | 4000    |
| weighted avg | 0.91      | 0.89   | 0.89     | 4000    |

```
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt

cm = confusion_matrix(all_labels, preds)

plt.figure()
plt.imshow(cm, interpolation='nearest', cmap='Blues') # Blue color map
plt.title("Confusion Matrix")
plt.colorbar()
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
import numpy as np

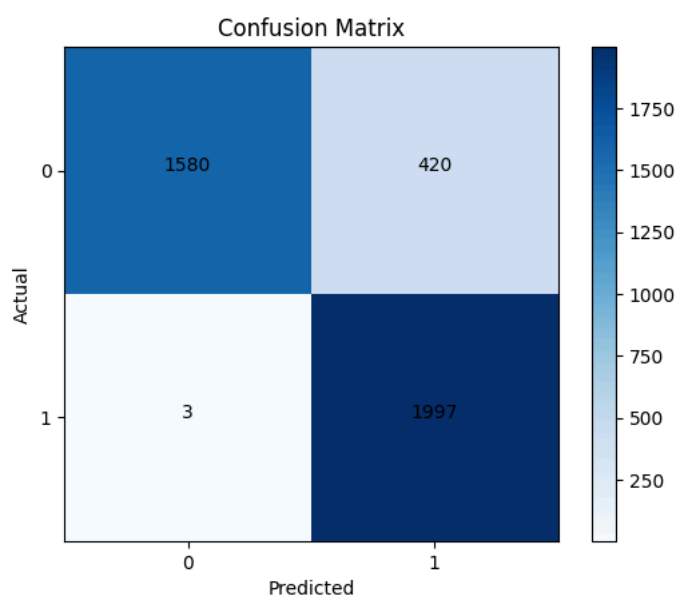
cm = confusion_matrix(all_labels, preds)

plt.figure()
plt.imshow(cm, interpolation='nearest', cmap='Blues')
plt.title("Confusion Matrix")
plt.colorbar()

tick_marks = np.arange(len(cm))
plt.xticks(tick_marks)
plt.yticks(tick_marks)

# Add numbers inside cells
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        plt.text(j, i, cm[i, j],
                 horizontalalignment="center",
                 color="black")

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
class TemperatureScaler(nn.Module):
    def __init__(self):
        super().__init__()
        self.temperature = nn.Parameter(torch.ones(1) * 1.5)

    def forward(self, logits):
```

```
return logits / self.temperature
```

```
scaler = TemperatureScaler().to(device)
optimizer = torch.optim.LBFGS([scaler.temperature], lr=0.01, max_iter=50)

logits_tensor = torch.tensor(all_logits, dtype=torch.float).to(device)
labels_tensor = torch.tensor(all_labels, dtype=torch.float).to(device)

def eval():
    optimizer.zero_grad()
    scaled_logits = scaler(logits_tensor)
    loss = loss_fn(scaled_logits, labels_tensor)
    loss.backward()
    return loss

optimizer.step(eval)
print("Calibrated temperature:", scaler.temperature.item())
```

Calibrated temperature: 1.234323263168335

```
df = pd.DataFrame({
    "confidence": probs,
    "label": all_labels,
    "prediction": preds
})

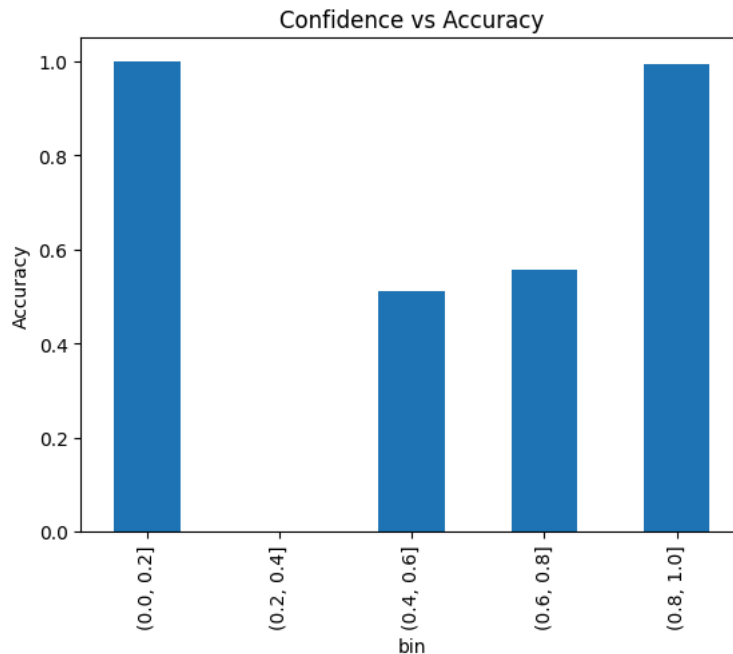
df["correct"] = df["prediction"] == df["label"]
```

```
bins = np.linspace(0, 1, 6)
df["bin"] = pd.cut(df["confidence"], bins)

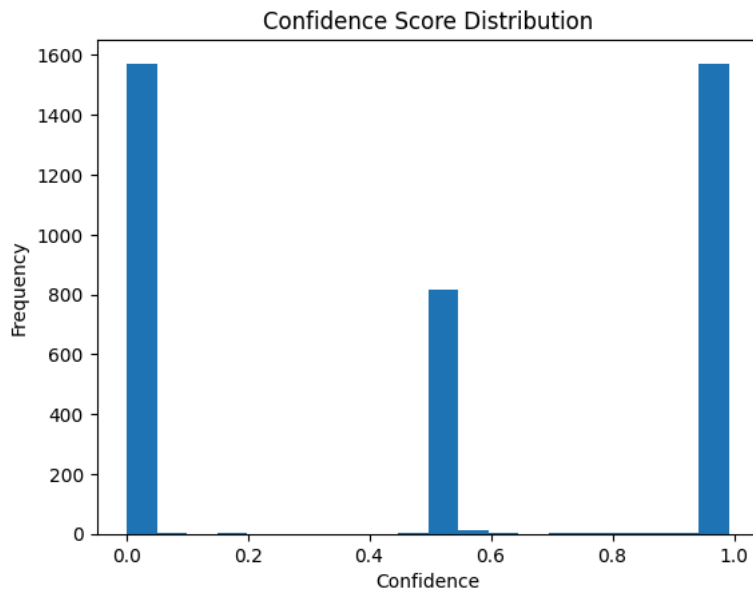
accuracy_by_bin = df.groupby("bin")["correct"].mean()

accuracy_by_bin.plot(kind="bar")
plt.title("Confidence vs Accuracy")
plt.ylabel("Accuracy")
plt.show()
```

/tmp/ipython-input-25157103.py:4: FutureWarning: The default of observed=False is deprecated and will be changed to True in  
accuracy\_by\_bin = df.groupby("bin")["correct"].mean()



```
plt.hist(df["confidence"], bins=20)
plt.title("Confidence Score Distribution")
plt.xlabel("Confidence")
plt.ylabel("Frequency")
plt.show()
```



```
high_conf_errors = df[
    (df["confidence"] > 0.8) &
    (df["correct"] == False)
]

print("High-confidence errors:")
print(high_conf_errors.head(10))
```

High-confidence errors:

|      | confidence | label | prediction | correct | bin        |
|------|------------|-------|------------|---------|------------|
| 86   | 0.988652   | 0.0   | 1          | False   | (0.8, 1.0] |
| 102  | 0.991839   | 0.0   | 1          | False   | (0.8, 1.0] |
| 825  | 0.991876   | 0.0   | 1          | False   | (0.8, 1.0] |
| 1889 | 0.991782   | 0.0   | 1          | False   | (0.8, 1.0] |
| 2035 | 0.989281   | 0.0   | 1          | False   | (0.8, 1.0] |
| 2180 | 0.991869   | 0.0   | 1          | False   | (0.8, 1.0] |
| 2314 | 0.987892   | 0.0   | 1          | False   | (0.8, 1.0] |
| 2405 | 0.991621   | 0.0   | 1          | False   | (0.8, 1.0] |
| 3010 | 0.991875   | 0.0   | 1          | False   | (0.8, 1.0] |
| 3903 | 0.991937   | 0.0   | 1          | False   | (0.8, 1.0] |

```
import gradio as gr
import matplotlib.pyplot as plt

# Dummy verification function (replace with your model)
def verify_answer(question, context, answer):
    if answer.lower() in context.lower():
        confidence = 0.9
        result_text = "Answer appears correct based on context."
    else:
        confidence = 0.3
        result_text = "Answer not supported by the context."

    return result_text, confidence

def verify_and_visualize(question, context, answer):
    result_text, confidence = verify_answer(question, context, answer)

    # Create confidence gauge
    fig, ax = plt.subplots(figsize=(4, 2))
    ax.barh(["Confidence"], [confidence])
    ax.set_xlim(0, 1)
    ax.set_xlabel("Score")
    ax.set_title("Confidence Gauge")
    plt.close(fig)

    # Reliability label
    if confidence > 0.75:
        label = "Highly Reliable"
        color = "green"
    elif confidence > 0.5:
        label = "Moderately Reliable"
        color = "orange"
    else:
        label = "Low Confidence"
```



```

        color = "red"

    status_html = f"""
    <div style="padding:10px;border-radius:10px;background-color:{color};color:white;font-weight:bold;text-align:center">
        {label}
    </div>
    """

    return result_text, fig, status_html

examples = [
    [
        "When did India gain independence?",
        "India gained independence from British rule on 15 August 1947.",
        "15 August 1947"
    ],
    [
        "Who discovered gravity?",
        "Isaac Newton formulated the laws of motion and gravity.",
        "Isaac Newton"
    ]
]

custom_css = """
body {background-color: #0f172a;}
.gradio-container {font-family: Arial, sans-serif;}
h1, h2, h3 {color: #38bdf8;}
"""

with gr.Blocks(css=custom_css) as demo:
    gr.Markdown("""
    # Confidence-Aware Answer Verification System
    ### Transformer-based Reliability and Explainability Engine
    """)

    with gr.Row():
        with gr.Column():
            question = gr.Textbox(label="Question")
            context = gr.Textbox(label="Context", lines=4)
            answer = gr.Textbox(label="Proposed Answer")

            verify_btn = gr.Button("Verify Answer")

        with gr.Column():
            status_box = gr.HTML(label="Reliability Level")
            confidence_plot = gr.Plot(label="Confidence Gauge")
            result_output = gr.Markdown(label="Detailed Result")

    verify_btn.click(
        verify_and_visualize,
        inputs=[question, context, answer],
        outputs=[result_output, confidence_plot, status_box]
    )

    gr.Examples(
        examples=examples,
        inputs=[question, context, answer]
    )

demo.launch()
```

```
/tmp/ipython-input-2703301738.py:66: DeprecationWarning: The 'css' parameter in the Blocks constructor will be removed in Gr
    with gr.Blocks(css=custom_css) as demo:
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`. Automatically setting `share

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://a994462104fde2c1cb.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the
```